

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

КУРСОВАЯ РАБОТА

Дисциплина: Объектно-ориентированное программирование
Тема: «Приложение с графическим интерфейсом для лабораторных работ 1–4»

Выполнил студент гр. в5130904/40321

(подпись)

Ф. А. Хороших

Руководитель, ст. преп.

(подпись)

А. П. Маслаков

«_____» _____ 2026 г.

Санкт-Петербург
2026

Оглавление

Постановка задачи.....	3
Описание реализации.....	5
Диаграмма классов.....	6
Задание 1. Передвижения героя.....	7
Задание 2. Аннотированные методы.....	9
Задание 3. Переводчик.....	10
Задание 4. Stream API	13
Перечень выполненных работ	14
Выводы	15

Постановка задачи

Цель курсовой работы — разработать приложение с графическим интерфейсом для заданий из лабораторных работ 1–4 по дисциплине «Объектно-ориентированное программирование»:

- герой (класс Hero) перемещается между двумя точками различными способами; реализовать выбор и смену способа перемещения в ходе выполнения программы с использованием паттерна «стратегия»;
- своя аннотация с целочисленным параметром; вызов из другого класса всех аннотированных защищённых и приватных методов столько раз, сколько указано в параметре аннотации, независимо от количества и типов их параметров;
- программа-переводчик: словарь из файла вида «слово или выражение | перевод», перевод по правилу максимальной длины левой части совпадения, собственные исключения при ошибках формата и чтения файла;
- шесть методов, реализованных только средствами Stream API: среднее значение списка, список строк в верхнем регистре с префиксом «_new_», квадраты элементов, встречающихся один раз, последний элемент коллекции с выбросом исключения на пустой коллекции, сумма чётных чисел массива, отображение строк в Map по первому символу.

Определены следующие задачи:

- реализовать выбор задания из списка, ввод входных данных и его выполнение;
- вывод результатов заданий 1–4 направить в текстовые области, защищённые от редактирования;
- для задания 3 предусмотреть выбор файлов словаря и текста для перевода, а также ручной ввод текста;
- для задания 4 предусмотреть ввод входных данных для методов;

- привести диаграмму классов реализованного приложения;
- оформить перечень выполненных работ.

Описание реализации

Приложение реализовано на языке Java 21 с использованием фреймворка JavaFX 21, сборка выполняется средствами Maven. Интерфейс построен непосредственно на Java-коде, без файлов разметки *.fxml.

Переключение между заданиями реализовано через элемент TabPane — каждому заданию соответствует своя вкладка (Task1Pane...Task4Pane) и собственное текстовое поле вывода, защищённое от редактирования (TextArea.setEditable(false)).

Логика всех четырёх заданий перенесена из соответствующих лабораторных работ без изменений: в лабораторных работах вывод уже строился через функциональный интерфейс Consumer<String>, а не напрямую в стандартный поток вывода, поэтому для отображения результата в текстовом поле оказалось достаточно подключить к этому интерфейсу метод TextArea.appendText — специальной подмены стандартного потока вывода не потребовалось.

Полный исходный код — в репозитории github.com/FedorKhoroshikh/oop-coursework (ветка main). Структура пакетов:

- app — точка входа (CourseworkApp, Launcher);
- ui — четыре панели JavaFX, по одной на задание (Task1Pane...Task4Pane);
- task1, task1.movement — герой и паттерн «стратегия»;
- task2 — аннотация @Repeat и рефлексивный вызов методов;
- task3 — словарь, токенизатор и переводчик;
- task4 — шесть методов на Stream API.

Диаграмма классов

На диаграмме показаны точка входа приложения, четыре представления JavaFX и перенесённые без изменений классы заданий 1–4. Сплошная стрелка означает использование или хранение ссылки, пунктирная с треугольником — реализацию интерфейса (Walking, HorseRiding и Flying реализуют MovementStrategy), пунктирная со стрелкой — выброс исключения.

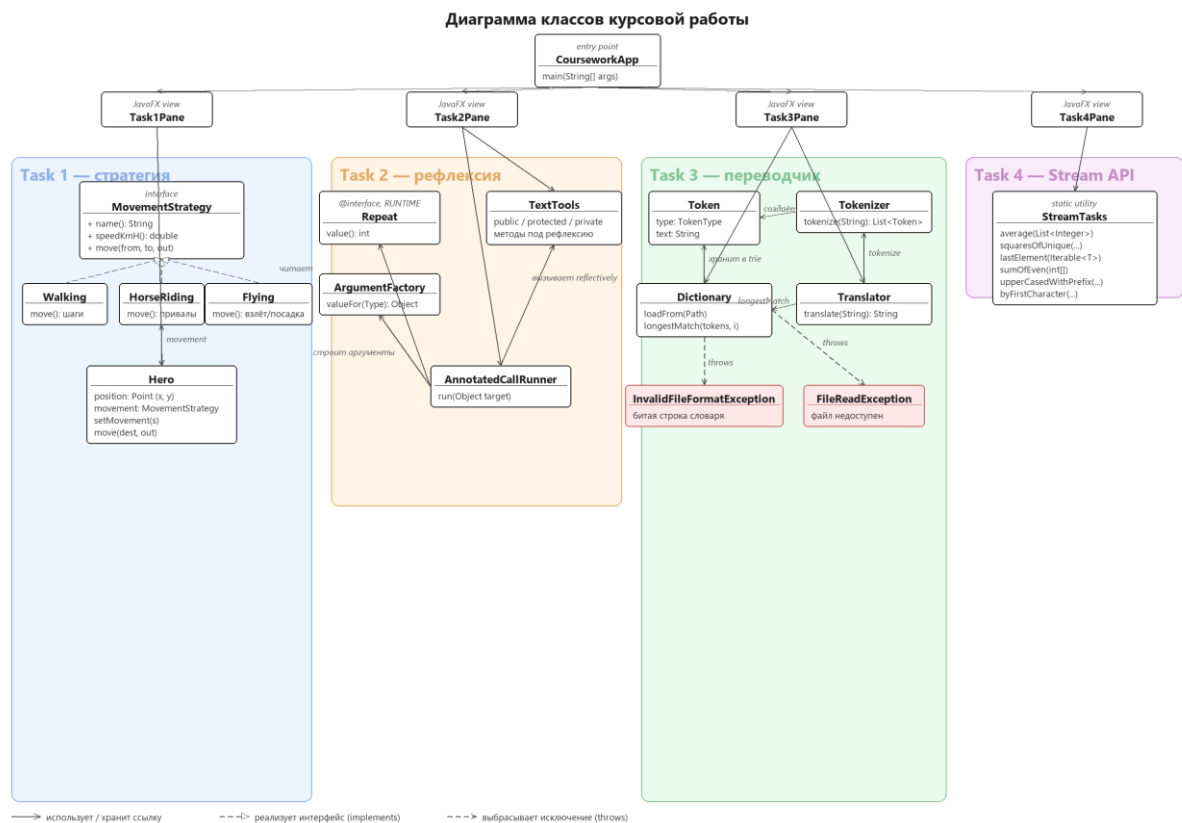


Рисунок 1. Диаграмма классов курсовой работы.

Задание 1. Передвижения героя

При реализации задания применён паттерн «стратегия»: класс Hero хранит ссылку на интерфейс MovementStrategy и делегирует ему перемещение, не зная, идёт герой пешком, едет верхом или летит. Способ перемещения выбирается в выпадающем списке и меняется методом Hero.setMovement() прямо в ходе выполнения программы — без пересоздания героя.

Перемещение показано графически на поле: пользователь либо кликает по полю мышью, либо вводит точные координаты в текстовые поля — оба способа сводятся к одному и тому же вызову Hero.move(). Каждая поездка рисуется отрезком, цвет которого соответствует выбранному способу перемещения (легенда справа от поля), а положение героя отмечено точкой. Кнопка «Move» перемещает героя выбранным способом к указанным координатам, кнопка «Compare all three ways» прогоняет один и тот же маршрут всеми тремя способами подряд и возвращает исходный способ перемещения. Параллельно результат каждой поездки выводится и в текстовую область — тем же способом, что и в консольной лабораторной работе.

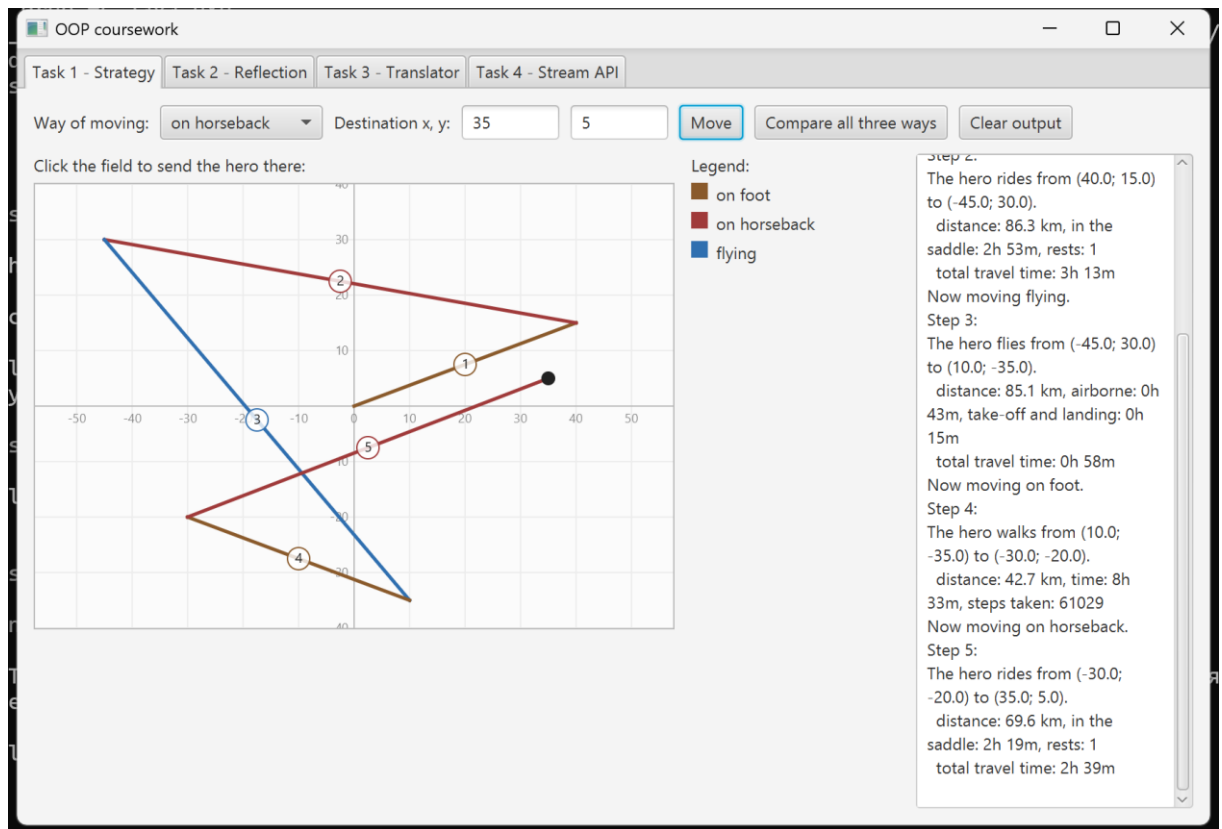


Рисунок 2. Вкладка «Задание 1».

Задание 2. Аннотированные методы

Аннотация `@Repeat(int value)` хранится с политикой `RUNTIME` — иначе JVM отбросила бы её при загрузке класса и рефлексия не увидела бы аннотацию вовсе. Класс `AnnotatedCallRunner` находит объявленные методы целевого объекта, оставляет только аннотированные `@Repeat` методы с модификатором `protected` или `private` (публичный аннотированный метод и неаннотированные методы нужного уровня доступа сознательно пропускаются) и вызывает каждый столько раз, сколько указано в аннотации.

Вызывающий код не зависит ни от количества, ни от типов параметров вызываемых методов: аргументы строит класс `ArgumentFactory` по отражённым типам параметров метода — для примитивов, строк, коллекций с реальным типом элемента, собственных классов и `record`, а для произвольного интерфейса создаётся динамический `Proxy`. Значение `null` нигде не передаётся.

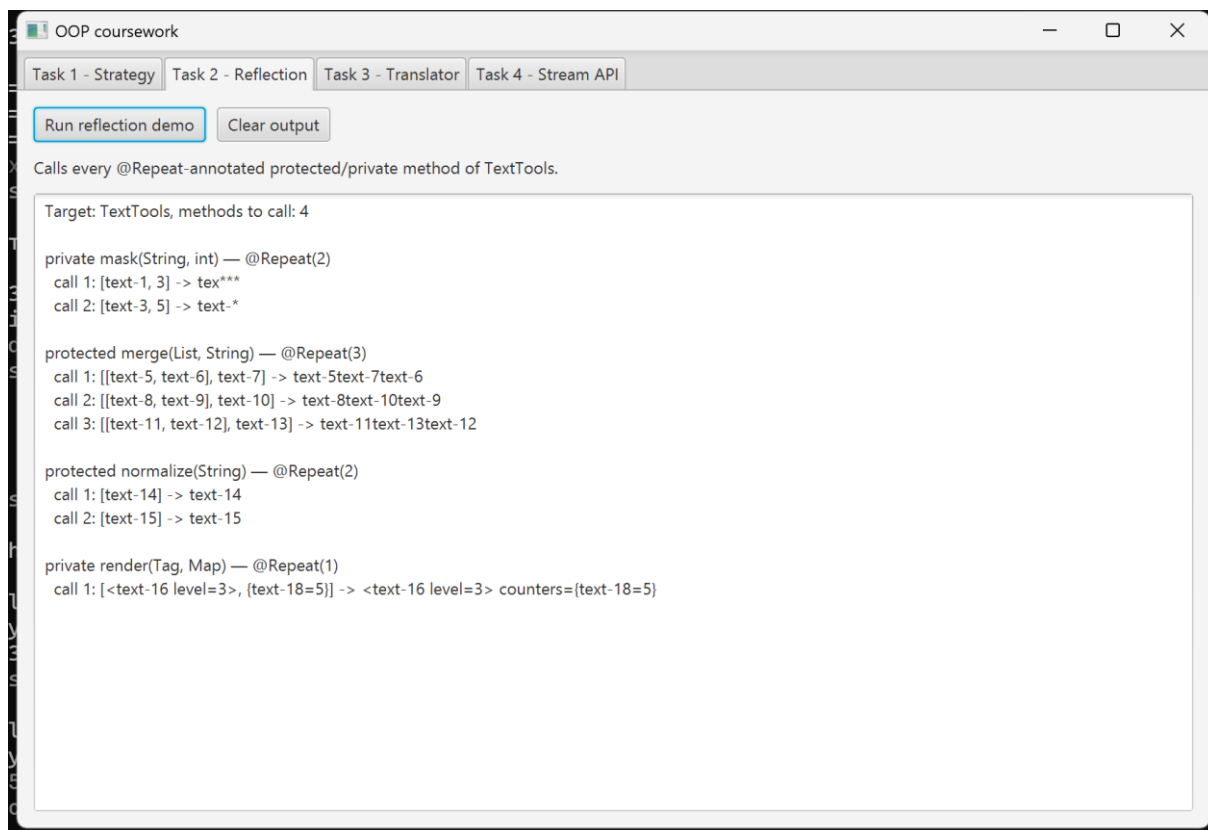


Рисунок 3. Вкладка «Задание 2».

Задание 3. Переводчик

Словарь читается из файла, выбранного через стандартный диалог FileChooser, и разбирается в дерево (trie) по словам фразы — это позволяет для каждого слова текста находить самую длинную подходящую фразу словаря напрямую, без перебора всех его записей. Текст для перевода можно как ввести вручную, так и загрузить из файла через отдельный диалог с последующим редактированием.

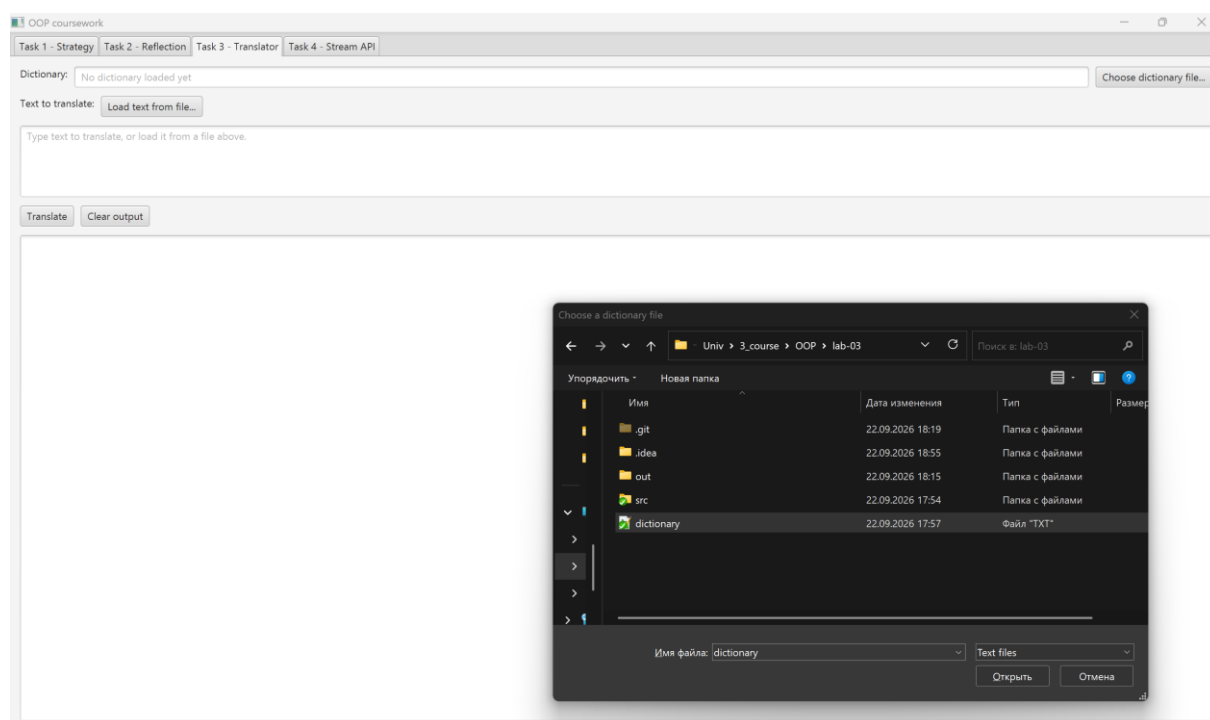


Рисунок 4. Выбор файла словаря.

Некорректный формат строки словаря приводит к исключению `InvalidFormatException`, недоступный файл — к `FileNotFoundException`; оба обрабатываются интерфейсом и выводятся пользователю текстом ошибки, не прерывая работу приложения.

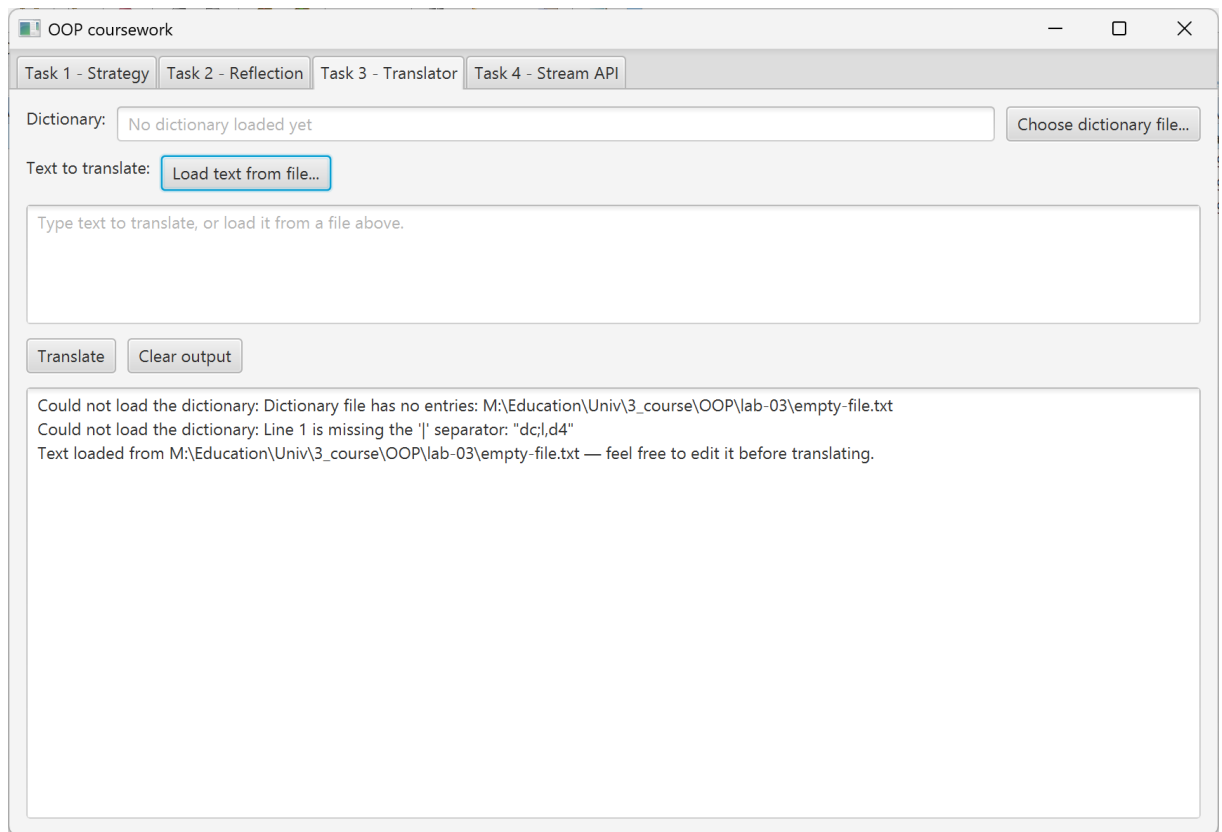


Рисунок 5. Пустой файл словаря и строка без разделителя «|» — оба случая пойманы и показаны текстом ошибки, приложение продолжает работать.

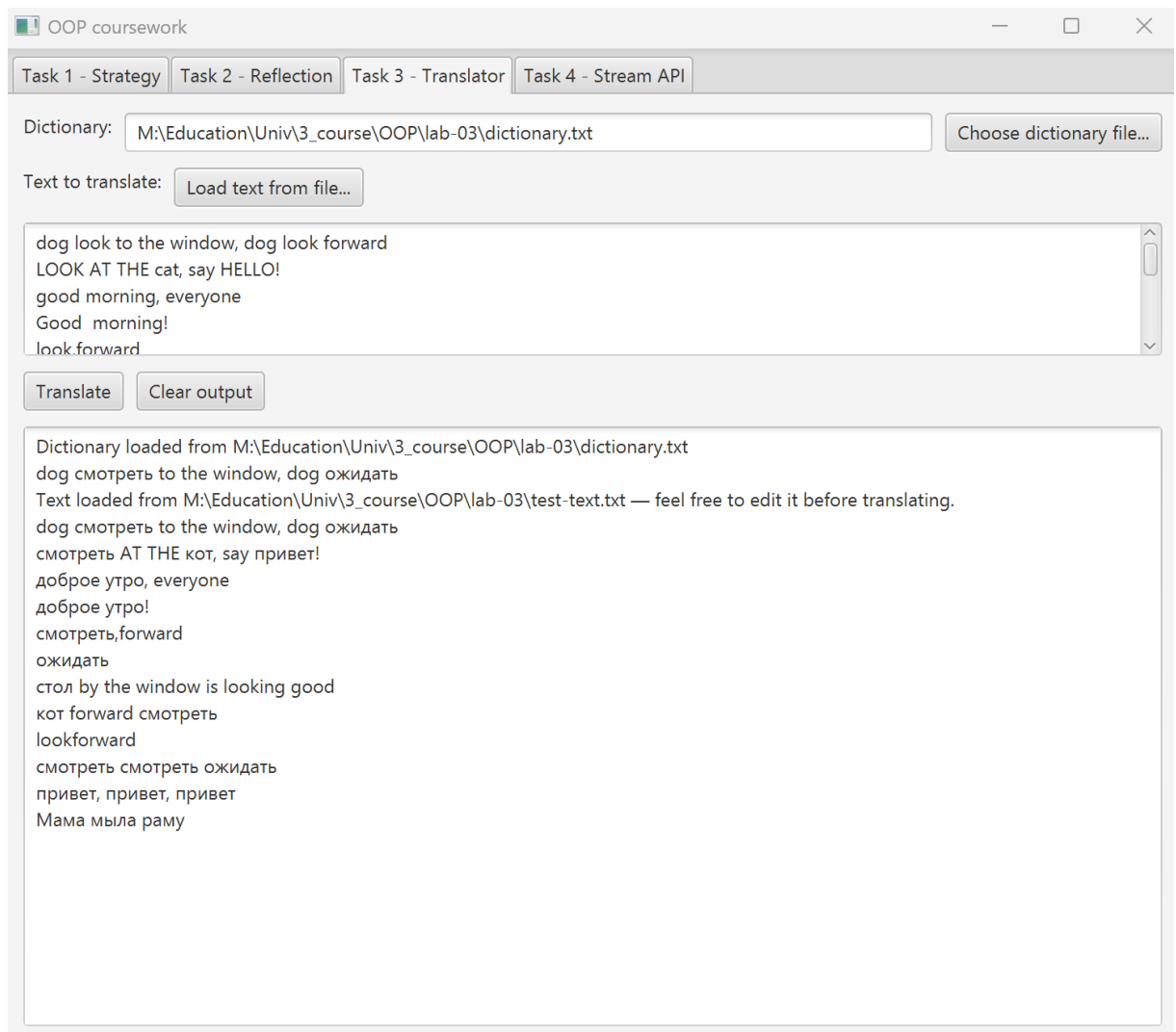


Рисунок 6. Результат перевода нескольких строк.

Задание 4. Stream API

Все шесть методов класса `StreamTasks` реализованы исключительно через Stream API, без ручных циклов и вручную заполняемых коллекций. Подсчёт «элементов, встречающихся один раз» выполнен через `groupingBy/counting`, а не через `distinct()`, который различает значения, но не считает число их повторений. Метод `lastElement` работает с произвольным `Iterable` через `StreamSupport` и выбрасывает исключение на пустой коллекции; `sumOfEven` на массиве без чётных чисел естественным образом возвращает 0 благодаря семантике `IntStream.sum()`.

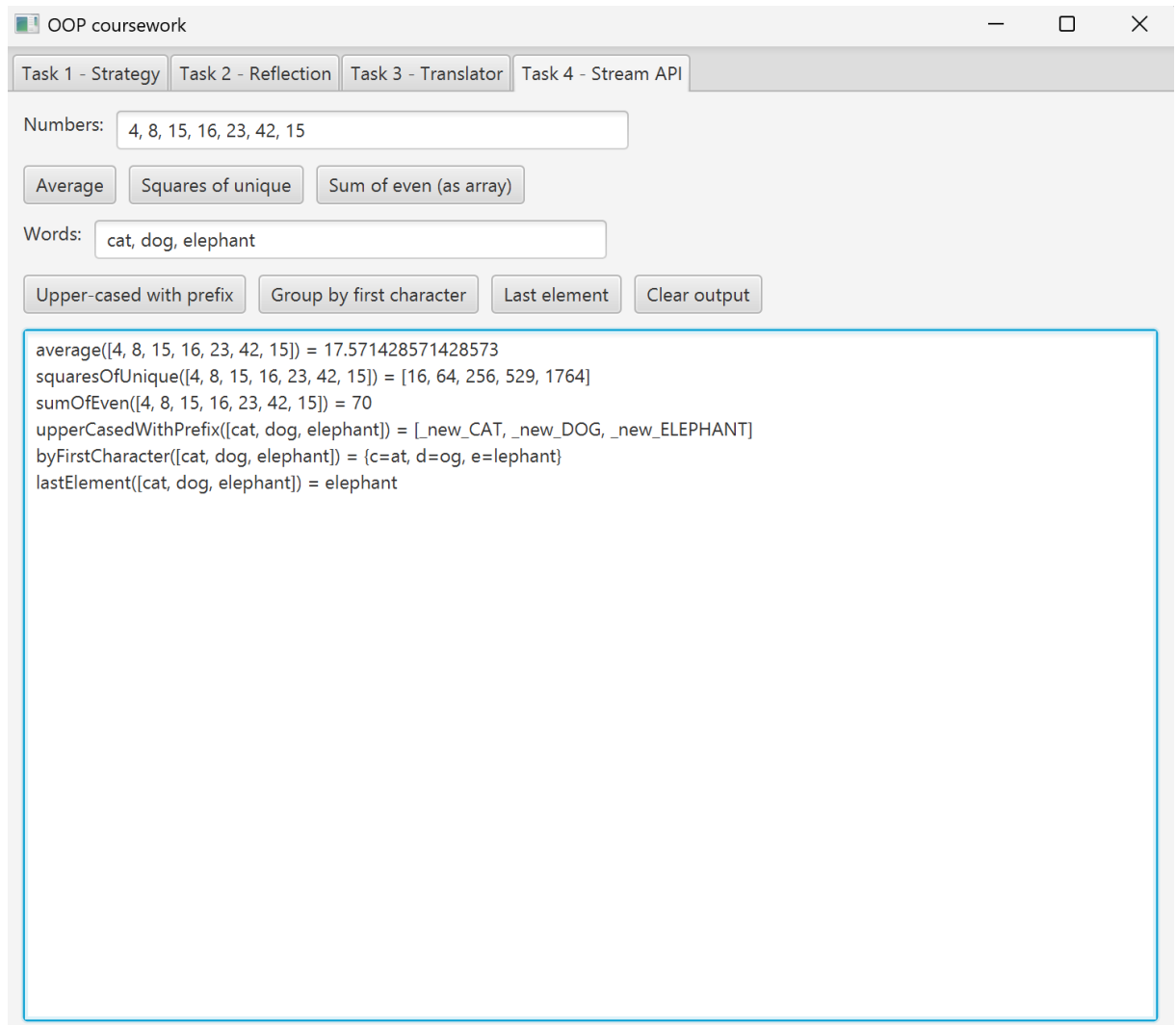


Рисунок 7. Вкладка «Задание 4».

Перечень выполненных работ

— Лабораторная работа 1 — паттерн «стратегия».

github.com/FedorKhoroshikh/oop-lab-01

— Лабораторная работа 2 — аннотации и рефлексия.

github.com/FedorKhoroshikh/oop-lab-02

— Лабораторная работа 3 — программа-переводчик.

github.com/FedorKhoroshikh/oop-lab-03

— Лабораторная работа 4 — методы Stream API.

github.com/FedorKhoroshikh/oop-lab-04

— Курсовой проект — приложение с графическим интерфейсом

JavaFX, объединяющее задания 1–4. github.com/FedorKhoroshikh/oop-coursework

Выводы

В ходе выполнения курсовой работы реализовано приложение на языке Java с графическим интерфейсом JavaFX, объединяющее задания четырёх лабораторных работ. Логика заданий перенесена без изменений — вывод результатов в них изначально строился через функциональный интерфейс, а не напрямую в стандартный поток, что и позволило подключить к нему текстовые области без переработки самих заданий. Для задания 3 реализован выбор файлов словаря и текста для перевода и ручной ввод текста; для задания 4 — ввод входных данных для методов. Поставленные задачи выполнены, цель курсовой работы достигнута.